Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. The effect of individual design choices is analyzed using training/validation curves, and a suitable configuration is finally selected using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, the following outcomes were achieved:

- explained different weight initialization strategies and demonstrated them experimentally;
- identified and analyzed overfitting using training and validation curves;
- compared SGD, Momentum, RMSProp and Adam under identical settings;
- explained CNN hyperparameters such as kernel size, stride, padding and pooling;
- verified Batch Normalization numerically and measured its effect on convergence;
- performed transfer learning and fine-tuning using MobileNetV2;
- tuned the important training hyperparameters one variable at a time;
- applied 5-fold cross-validation for model selection; and
- justified the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup and Source Code

Source Code: https://github.com/D-Rockie/Deep-Learning-Lab-2026.git

Dataset: Oxford-IIIT Pet Dataset. The dataset contains 7,349 RGB images of cats and dogs belonging to 37 breeds, with different spatial dimensions. All images were resized to

$$224 \times 224 \times 3$$

and normalized to $[-1, 1]$, exactly the preprocessing required by the ImageNet-pretrained MobileNetV2. The two official Oxford archives (`images.tar.gz`, `annotations.tar.gz`) were read directly from the Oxford mirror, so the experiment does not depend on `tensorflow_datasets`.

Splits actually used.

Split	Images	Role
Training pool (official `trainval`)	3,680	used for all training and for 5-fold CV
↪ train (80%)	2,944	single-run studies (Sections 5–9)
↪ validation (20%)	736	single-run studies (Sections 5–9)
Test set (official `test`)	3,669	**untouched** until Section 12
Total	7,349	37 breeds

The split is stratified: 74–80 images per class in training (mean 79.6) and 19–20 per class in validation (mean 19.9). Memory occupied by the materialised 224×224 uint8 arrays was 1.11 GB. The random-guess (chance) accuracy is $1/37 = 2.70\,\%$, which is the reference level for the zero-initialization run in Section 5.

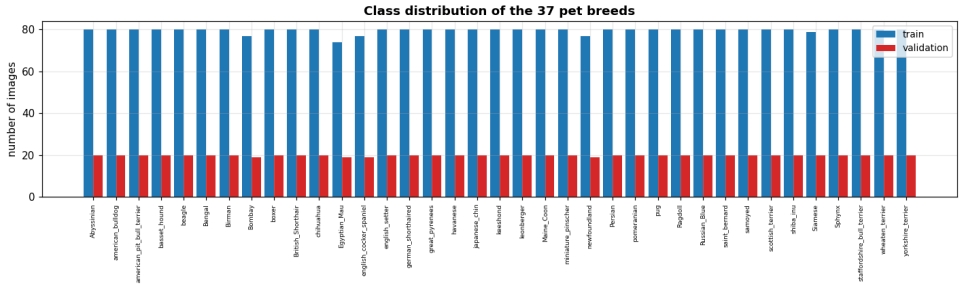


Figure 1: Class distribution of the training pool over the 37 breeds. The dataset is close to balanced, so plain accuracy is a meaningful metric and no class re-weighting is required.

Figure 2: One representative sample per class after resizing to 224×224. Several breed pairs (e.g. the terriers, the spotted cats) are visually very similar, which anticipates the confusion structure observed in Section 12.

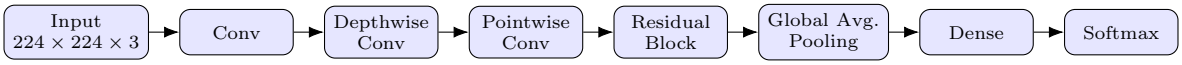
Experimental considerations enforced: RGB images resized to 224×224; inputs normalized as required by the pretrained model; separate training, validation and test data; the test set kept untouched during all hyperparameter selection; and MobileNetV2 used because it is computationally far lighter than large CNN architectures.

4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation. Its important components are:

- **depthwise convolution** – one $k \times k$ filter per input channel, with no cross-channel mixing;
- **pointwise** 1×1 **convolution** – mixes channels, no spatial extent;
- **inverted residual blocks** – expand → depthwise → project, with a skip connection;
- **linear bottlenecks** – no ReLU on the narrow projection output;
- **batch normalization** after every convolution; and
- **ReLU6** activation, $\min(\max(0, x), 6)$, which is quantisation friendly.

A simplified representation is:



Note: the diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

4.1 Measured architecture (ImageNet weights, `include_top=False`)

Property	Value
Total layers	154
Total parameters	2,257,984
Output shape	(None, 7, 7, 1280)
BatchNormalization layers	52
Conv2D layers	35
ReLU layers	35
DepthwiseConv2D layers	17
Add (residual) layers	10
ZeroPadding2D layers	4

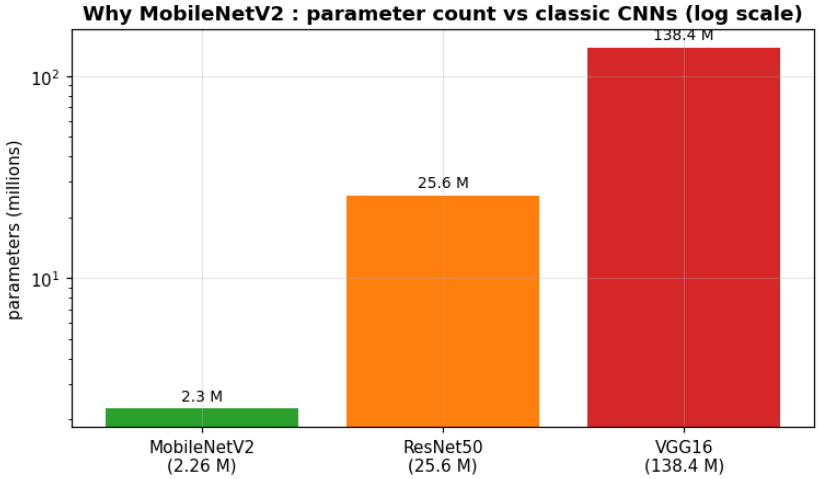


Figure 3: Parameter count of MobileNetV2 compared with two classic ImageNet CNNs.

- **What the plot shows:** Parameter counts of MobileNetV2 against two classic ImageNet CNNs.
- **Trend observed:** MobileNetV2 needs roughly 11× fewer parameters than ResNet50 and 61× fewer than VGG16.
- **Why it occurs:** Depthwise separable convolution factorises a $K \times K \times C_{in} \times C_{out}$ kernel into a $K \times K \times C_{in}$ depthwise part plus a $1 \times 1 \times C_{in} \times C_{out}$ pointwise part, cutting cost by about $1/K^2$.

4.2 Cost of depthwise separable convolution (measured)

Feature map	C_{in}	C_{out}	Std. params	DW-sep. params	Param saving	FLOP saving
112×112	32	64	18,432	2,336	7.9×	7.9×
56×56	128	128	147,456	17,536	8.4×	8.4×
28×28	256	256	589,824	67,840	8.7×	8.7×
14×14	512	512	2,359,296	266,752	8.8×	8.8×
7×7	960	320	2,764,800	315,840	8.8×	8.8×

The theoretical ratio is

$$\frac{K^2 C_{in} + C_{in} C_{out}}{K^2 C_{in} C_{out}} = \frac{1}{C_{out}} + \frac{1}{K^2},$$

which for $K = 3$ and large C_{out} tends to 1/9, i.e. an 8–9× reduction. This is exactly why MobileNetV2 has only ≈ 2.26 M parameters against ≈ 138 M for VGG16.

4.3 Experimental strategy: the frozen-base feature cache

Sections 5–9 require dozens of controlled training runs under the rule "change one hyperparameter at a time". All of these studies are performed in the feature-extraction regime, where the pretrained base is *frozen*, so the base returns exactly the same 1280-dimensional Global-Average-Pooled feature for an image on every run. Those features were therefore computed **once** and cached:

Feature dimension	1280
Frozen base parameters	2,257,984
Training pool features	$(3680, 1280)$ extracted in 98.0 s
Test set features	$(3669, 1280)$ extracted in 97.6 s
Cached memory	37.6 MB (against 1.11 GB of images)
Feature statistics	mean 0.311, std 0.508, min 0.000, max 5.529

Training the head on cached features is mathematically identical to training the full model with a frozen base (without augmentation) and reduces each controlled run from about a minute to about a second. Full end-to-end 224×224 models are still trained in Sections 10, 12 and 16, where the base is unfrozen and the cache no longer applies.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. Four strategies were applied to the classification head, with every other setting held fixed (Adam, $lr = 0.001$, batch = 32, no dropout, no L2, no BN, hidden = 256):

Strategy	Rule	Expected behaviour
Zero	$W = 0$	all hidden units compute the same thing; symmetry is never broken
Random	$W \sim N(0, 1)$	symmetry broken but variance far too large; exploding activations
Xavier/Glorot	$\mathrm{Var}(W) = 2/(\mathrm{fan}_{in} + \mathrm{fan}_{out})$	balances forward/backward variance; derived for tanh/sigmoid
He	$\mathrm{Var}(W) = 2/\mathrm{fan}_{in}$	factor 2 compensates for ReLU zeroing half the activations

Results

Initialization	Final train loss	Best val. accuracy (%)	Epoch to converge	Time (s)
Zero	3.6112	2.72	1	13.6
Random	0.0859	74.18	14	13.3
Xavier/Glorot	0.0010	91.03	9	13.3
He	**0.0010**	**91.71**	12	13.2

Best initialization: He, which is used for the rest of the experiment.

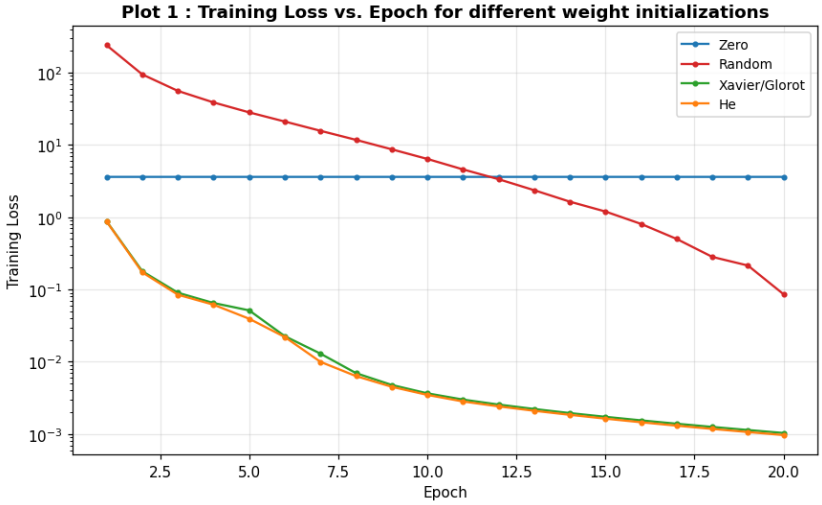


Figure 4: Plot 1: Training loss vs. epoch for the four initialization strategies.

- **What the plot shows:** Training-loss trajectory of the same head trained with four initialization schemes.
- **Trend observed:** He and Xavier fall fastest and lowest; Random ($\sigma = 1$) starts orders of magnitude higher and decreases erratically; Zero initialization is essentially flat at $\ln 37 \approx 3.61$.
- **Why it occurs:** He/Xavier keep the layer-wise activation variance close to 1 so gradients are well scaled. Large random weights saturate and explode activations. With $W = 0$ every hidden unit is identical and $\partial L/\partial W_1 \propto W_2 = 0$, so the hidden layer never receives a gradient.

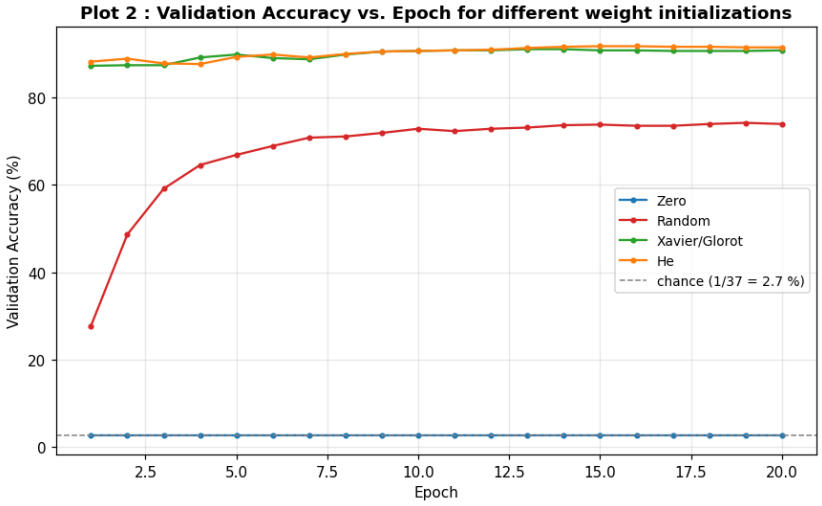


Figure 5: Plot 2: Validation accuracy vs. epoch for the four initialization strategies.

- **What the plot shows:** Validation accuracy per epoch for the four initialization strategies.
- **Trend observed:** He and Xavier converge within a few epochs to the highest accuracy (He marginally higher and smoother); Random is far below and noisy; Zero stays at the 2.70 % chance level.
- **Why it occurs:** Variance-scaled initialization gives a well-conditioned starting point; He's factor of 2 exactly compensates for ReLU discarding the negative half, which is why it is the best match for this ReLU head.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. The **generalization gap** is (train accuracy − validation accuracy). Four settings were compared, one change at a time (He init, Adam, $lr = 0.001$, batch $= 32$).

Variant	Train acc. (%)	Val. acc. (%)	Best val. acc. (%)	Gap (pp)	Best val-loss epoch	Time (s)
No regularization	100.00	91.44	**91.71**	**8.56**	9	13.6
L2 (10^{-3})	97.25	86.28	91.03	10.97	17	14.7
Dropout (0.5)	99.22	90.35	91.03	8.87	3	14.4
Batch Normalization	100.00	91.17	**91.71**	8.83	4	15.1

Overfitting check (does validation loss rise while training loss keeps falling?):

Variant	Min. val-loss epoch	Final val. loss	Verdict
No regularization	9	0.4002	overfitting after that epoch
L2 (10^{-3})	17	0.7526	overfitting after that epoch
Dropout (0.5)	3	0.3723	overfitting after that epoch
Batch Normalization	4	0.3366	overfitting after that epoch

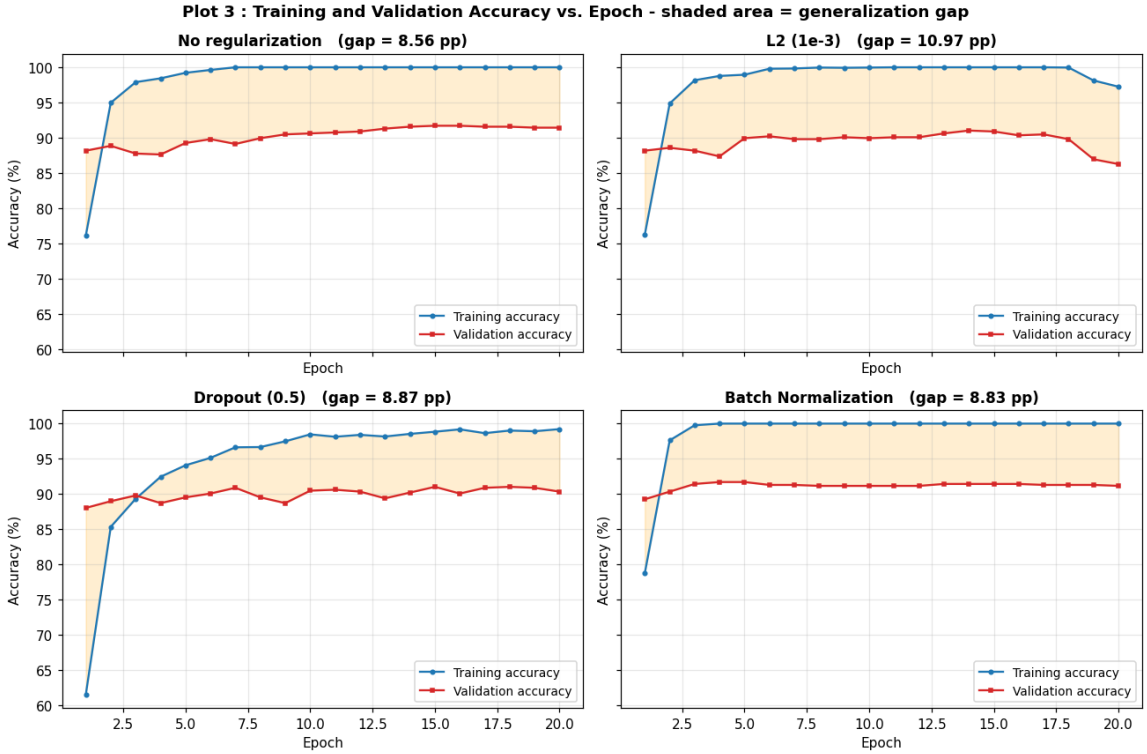


Figure 6: Plot 3: Training and validation accuracy vs. epoch for each regularization setting. The shaded band is the generalization gap.

- **What the plot shows:** Training vs. validation accuracy for four regularization settings; the shaded band is the generalization gap.
- **Trend observed:** The unregularized head drives training accuracy to 100 % while validation accuracy saturates near 91 %. L2 separates the two curves the most because it also lowers the training fit; BN converges fastest with an intermediate gap.
- **Why it occurs:** Dropout injects multiplicative noise and forces redundant features; L2 shrinks the weights and limits effective capacity, so both reduce memorisation of the training set. BN mainly stabilises and accelerates optimisation and only mildly regularises.

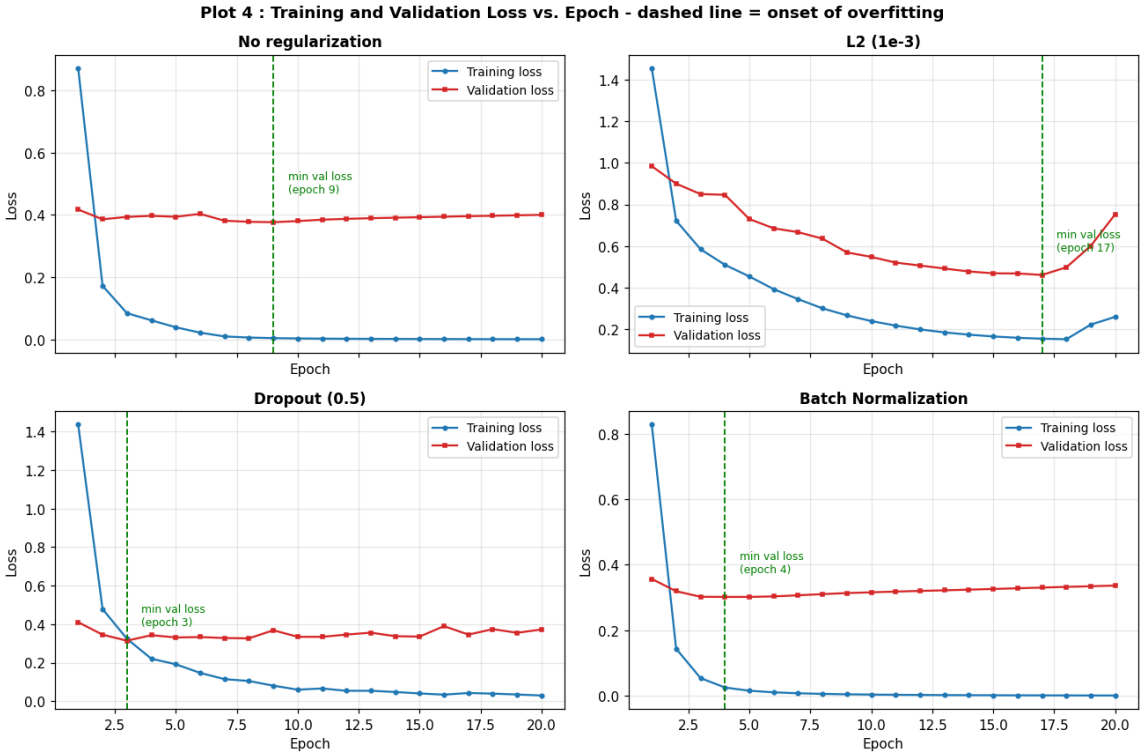


Figure 7: Plot 4: Training and validation loss vs. epoch. The dashed vertical line marks the minimum of the validation loss, i.e. the correct early-stopping point.

- **What the plot shows:** Training and validation loss per epoch for each regularization variant.
- **Trend observed:** For the unregularized model the training loss keeps falling while the validation loss reaches a minimum at epoch 9 and then rises — the classic overfitting signature. With L2 the minimum is pushed out to epoch 17.
- **Why it occurs:** Once the model starts fitting noise specific to the training images, the extra capacity helps only the training loss. Regularizers penalise exactly that behaviour, so the minimum of the validation loss (the correct early-stopping point) is pushed later.

How overfitting is identified from the curves: the moment the validation loss stops decreasing and starts increasing while the training loss keeps decreasing. The vertical distance between the training and validation accuracy curves in Plot 3 is the generalization gap.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training. For a mini-batch $x_1, x_2, \ldots, x_m$, the batch mean is

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

and the final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example (verified in code)

Consider $x = [2, 4, 6, 8]$. The mean is

$$\mu_B = \frac{2+4+6+8}{4} = 5,$$

the variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = \frac{9+1+1+9}{4} = 5,$$

and therefore $\sqrt{5} \approx 2.236$. Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342, \quad \hat{x}_2 = \frac{4-5}{2.236} \approx -0.447, \quad \hat{x}_3 = \frac{6-5}{2.236} \approx 0.447, \quad \hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence

$$\boxed{\hat{x} \approx [-1.342,\ -0.447,\ 0.447,\ 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Computed verification. The notebook reproduced $\mu_B = 5.0000$, $\sigma_B^2 = 5.0000$, $\sqrt{\sigma_B^2} = 2.2361$ and $\hat{x} = [-1.342, -0.447, 0.447, 1.342]$, with $\text{mean}(\hat{x}) = 0.000000$ and $\text{std}(\hat{x}) = 0.9999$. Keras' own `BatchNormalization` layer in training mode returned the identical vector.

Effect of the learnable parameters:

γ	β	Output y
1.0	0.0	[−1.342, −0.447, 0.447, 1.342]
2.0	0.0	[−2.683, −0.894, 0.894, 2.683]
1.0	3.0	[1.658, 2.553, 3.447, 4.342]
0.5	−1.0	[−1.671, −1.224, −0.776, −0.329]

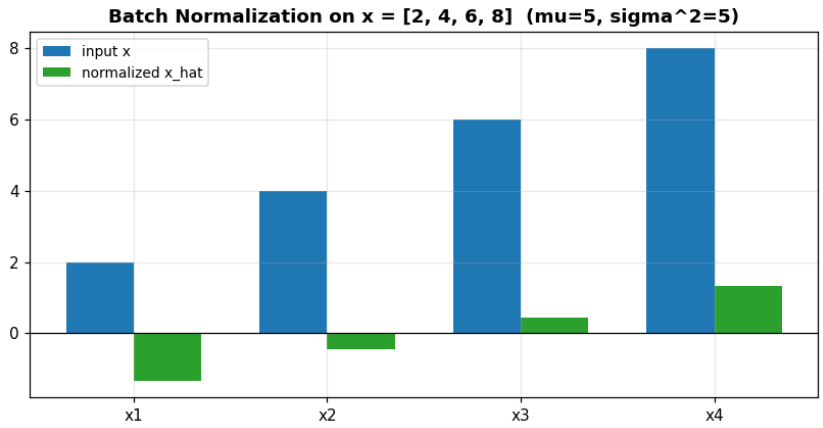


Figure 8: Visualisation of the numerical Batch Normalization example: the raw mini-batch $[2, 4, 6, 8]$ and its normalized counterpart.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift, and can even undo the normalization entirely.

With vs. Without Batch Normalization

Setting	Best val. accuracy (%)	Epoch to converge	Val. accuracy at epoch 3 (%)
Without BN	91.71	12	87.77
With BN	91.71	**3**	**91.44**

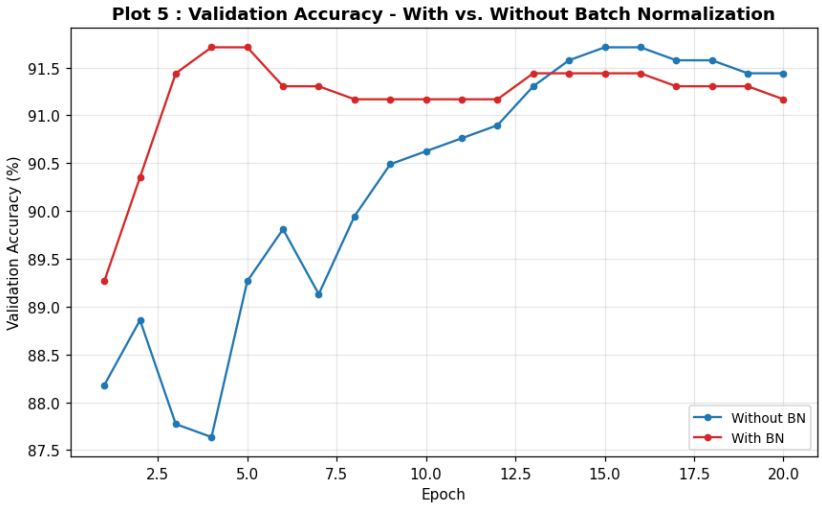


Figure 9: Plot 5: Validation accuracy vs. epoch with and without Batch Normalization.

- **What the plot shows:** Validation accuracy per epoch of the same head with and without a BatchNormalization layer between `Dense(256)` and ReLU.
- **Trend observed:** The BN curve rises much more steeply — it reaches 91.44 % by epoch 3, which the plain head only matches at epoch 12 — and is visibly smoother. The final accuracies are identical at 91.71 %.
- **Why it occurs:** BN re-centres and re-scales the pre-activations of every mini-batch, which removes internal covariate shift, keeps the units in the non-saturating region of ReLU and smooths the loss surface, allowing effectively larger stable steps. Its benefit here is convergence *speed and stability*, not raw final accuracy.

8. Optimization Algorithms

Optimizer	Update rule (idea)	Character
SGD	$w \leftarrow w - \eta g$	simplest, needs a well-tuned η, slow in ravines
Momentum	$v \leftarrow \beta v + g;\ w \leftarrow w - \eta v$	accumulates velocity, damps oscillations
RMSProp	$s \leftarrow \rho s + (1-\rho) g^2;\ w \leftarrow w - \eta g / \sqrt{s}$	per-parameter adaptive step
Adam	RMSProp + Momentum + bias correction	adaptive and momentum-driven; strong default

All four were run with the *same* learning rate 0.001, which is the honest one-variable-at-a-time comparison. Because plain SGD is known to need a larger step, two supplementary rows at $lr = 0.01$ are also reported.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	0.8497	84.24	19	10.2
Momentum	0.1066	89.95	9	11.3
RMSProp	0.0001	90.49	7	12.4
Adam	0.0010	**91.71**	12	13.6
SGD ($lr = 0.01$)	0.1060	89.81	7	10.2
Momentum ($lr = 0.01$)	0.0063	91.17	10	11.6

Best optimizer at the common learning rate: Adam.

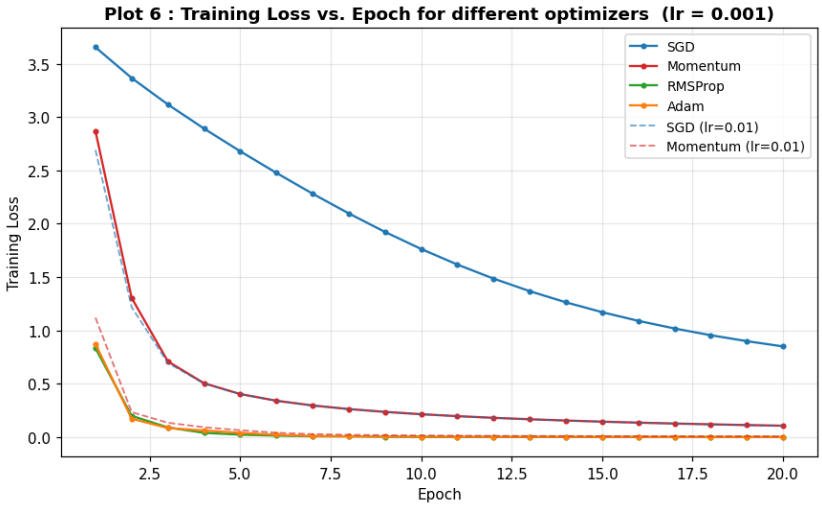


Figure 10: Plot 6: Training loss vs. epoch for the four optimizers.

- **What the plot shows:** Training-loss curves of SGD, Momentum, RMSProp and Adam at an identical learning rate.
- **Trend observed:** Adam and RMSProp drop steeply within 2–3 epochs; Momentum is clearly faster than plain SGD, which decreases almost linearly and is still at loss 0.85 after 20 epochs.
- **Why it occurs:** Adaptive methods rescale each coordinate by a running RMS of its gradient, so rarely-updated features still get large steps. Momentum accumulates a velocity that cancels oscillations across ravines. Plain SGD applies one global step size, which at $lr = 0.001$ is simply too small for this loss surface.

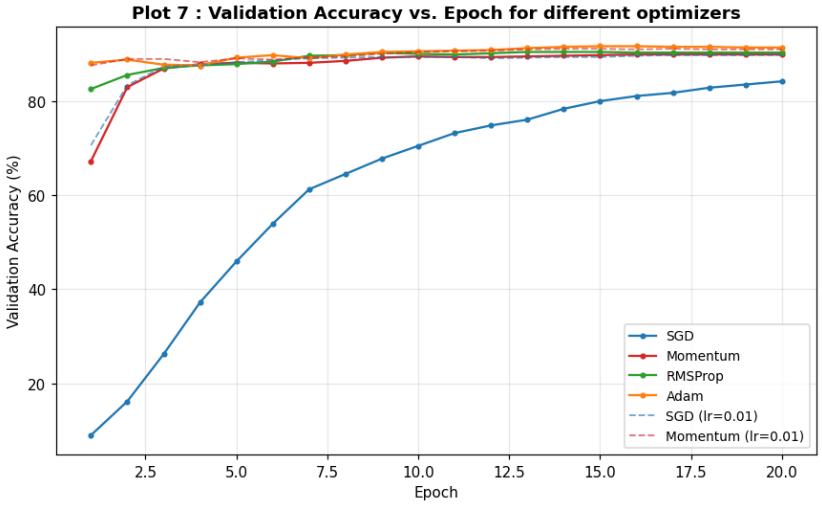


Figure 11: Plot 7: Validation accuracy vs. epoch for the four optimizers.

- **What the plot shows:** Validation accuracy per epoch for the four optimizers.
- **Trend observed:** Adam reaches its plateau first and highest (91.71 %), RMSProp is a close second but noisier, Momentum catches up slowly and plain SGD has not converged within the budget (84.24 %).
- **Why it occurs:** Adam combines a momentum term (first moment) with per-parameter adaptive scaling (second moment) and bias correction, so it converges fast without any learning-rate search — which is exactly why it is the default choice for transfer-learning heads.

9. CNN Hyperparameter Tuning

Hyperparameter	Values investigated	Where studied
Learning Rate	0.001, 0.0001	Plot 8
Batch Size	16, 32, 64	Plot 9
Dropout Rate	0, 0.25, 0.5	Plot 10
Optimizer	SGD, Adam	this section + Section 8
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$	Section 10
Frozen Layers	Frozen base / Partial unfreezing	Section 10

For a convolution operation, the output dimension is

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride. This was evaluated for the configurations actually used inside MobileNetV2:

N	K	P	S	O	Meaning
224	3	0	1	222	3×3, valid padding, stride 1
224	3	1	1	224	3×3, same padding, stride 1 (size preserved)
224	3	1	2	112	3×3, same padding, stride 2 (down-sample $\times 2$)
224	5	2	1	224	5×5, same padding, stride 1
224	7	3	2	112	7×7, stride 2 (stem-style)
112	3	1	2	56	3×3 depthwise, stride 2
56	1	0	1	56	1×1 pointwise (spatial size unchanged)
14	2	0	2	7	2×2 max-pooling, stride 2

- **kernel size K:** larger K gives a larger receptive field but a smaller output and more FLOPs;
- **padding P:** $P = (K-1)/2$ with $S = 1$ gives "same" padding and preserves the spatial size;
- **stride S:** $S = 2$ halves the spatial resolution, which is how MobileNetV2 down-samples;
- **pooling:** 2×2 / stride 2 also halves resolution but has no learnable weights.

Experimental rule (enforced): one hyperparameter is changed at a time, with the fixed baseline He init, Adam, $lr = 0.001$, batch = 32, dropout = 0.25, no L2, no BN.

Hyperparameter	Value	Best val. accuracy (%)
Learning Rate	0.001	91.30
Learning Rate	0.0001	**91.44**
Batch Size	16	91.17
Batch Size	32	91.30
Batch Size	64	**91.44**
Dropout Rate	0.00	**91.71**
Dropout Rate	0.25	91.30
Dropout Rate	0.50	91.03
Optimizer	SGD	83.97
Optimizer	Adam	**91.30**

Selected so far: $lr = 0.0001$, batch = 64, dropout = 0.00, optimizer = Adam, initialization = He.

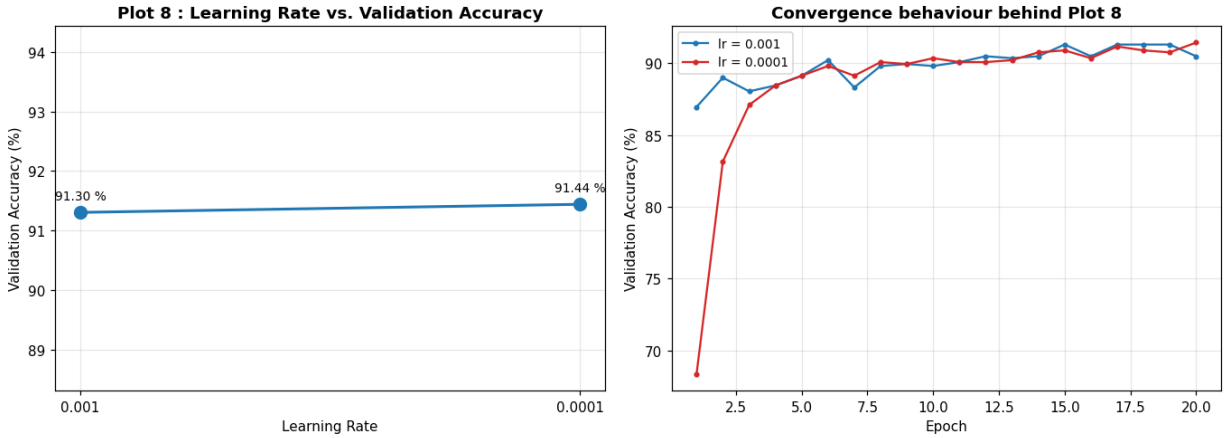


Figure 12: Plot 8: Learning rate vs. validation accuracy (left) and the underlying convergence curves (right).

- **What the plot shows:** Best validation accuracy as a function of the learning rate (0.001 vs. 0.0001), plus the underlying convergence curves.
- **Trend observed:** Both end within 0.15 pp of each other, but $lr = 0.001$ reaches its plateau within a few epochs while $lr = 0.0001$ climbs slowly and only just edges ahead at the end of the 20-epoch budget.
- **Why it occurs:** The learning rate scales every parameter update. A too-small step needs many more epochs to travel the same distance in weight space; a too-large step overshoots minima and can diverge.

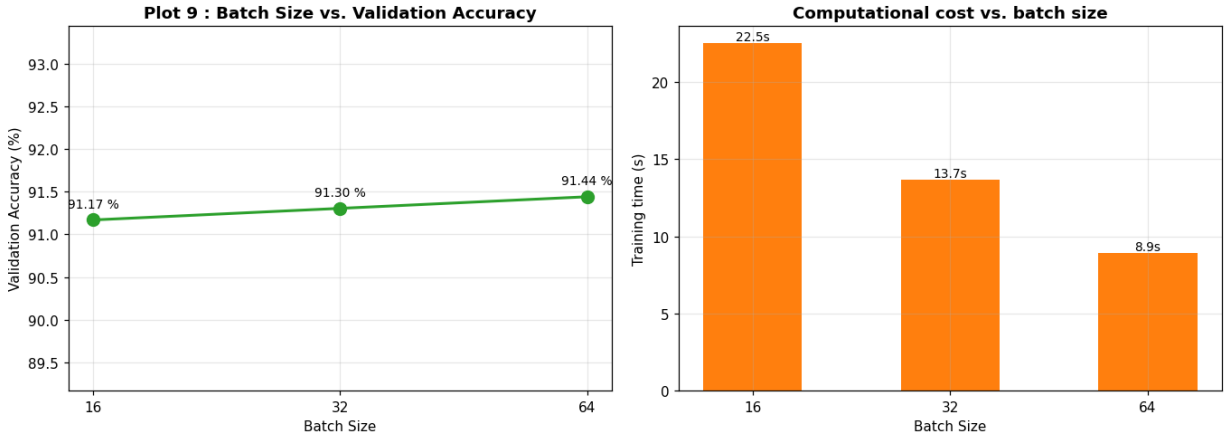


Figure 13: Plot 9: Batch size vs. validation accuracy (left) and training time (right).

- **What the plot shows:** Validation accuracy and wall-clock training time against batch size 16 / 32 / 64.
- **Trend observed:** Accuracy varies by under 0.3 pp across the three batch sizes (91.17 → 91.44 %), while training time falls from 22.5 s to 8.9 s as the batch grows.
- **Why it occurs:** A larger batch gives a lower-variance gradient estimate and better hardware utilisation, but it also performs fewer updates per epoch and the reduced gradient noise slightly weakens its implicit regularisation — the two effects roughly cancel in accuracy while the speed-up remains.

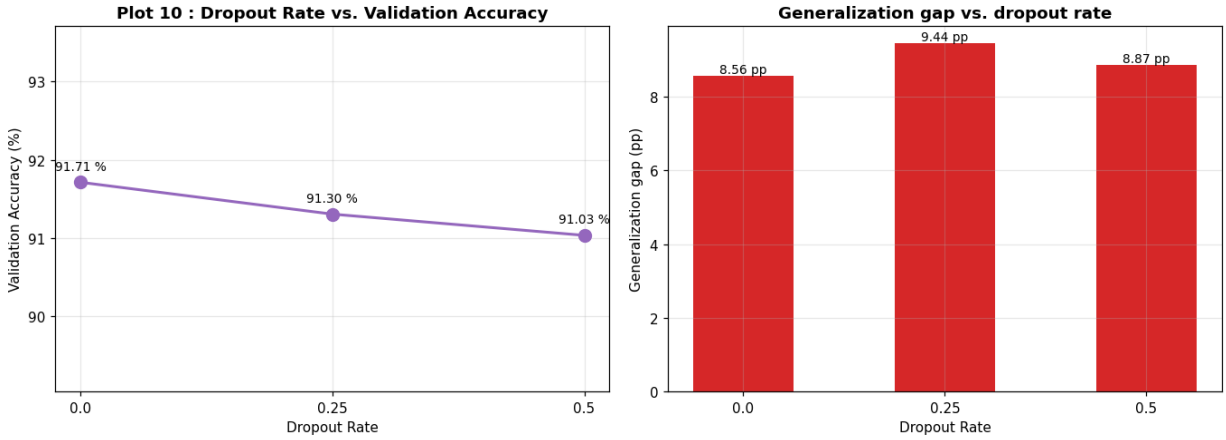


Figure 14: Plot 10: Dropout rate vs. validation accuracy (left) and vs. the generalization gap (right).

- **What the plot shows:** Validation accuracy and the train–validation gap as a function of the dropout rate.
- **Trend observed:** On the cached frozen features accuracy decreases slightly and monotonically with the dropout rate ($91.71 \rightarrow 91.03\,\%$), while the generalization gap is smallest at $p = 0$ and $p = 0.5$ and largest at $p = 0.25$.
- **Why it occurs:** Dropout randomly zeroes a fraction p of hidden units each step, which prevents co-adaptation and effectively averages an ensemble of thinned networks. Here the head is small and the frozen features are already strongly discriminative, so there is little to over-fit and dropout mainly removes usable capacity.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used. These runs are full end-to-end 224×224 trainings (the feature cache no longer applies once the base changes), with light data augmentation.

Case A: Feature Extraction

Pretrained MobileNetV2 $\rightarrow$ Freeze Base $\rightarrow$ New Classifier

All pretrained convolutional layers are frozen and only the newly added classification layers are trained (337,445 trainable / 2,257,984 frozen parameters).

Case B: Fine-Tuning

Pretrained MobileNetV2 $\rightarrow$ Unfreeze Selected Layers $\rightarrow$ Small Learning Rate

Standard two-phase recipe: the head is first warmed up with the base frozen, then the last 30 layers are unfrozen and training continues at $lr = 10^{-5}$ (1,848,165 trainable / 747,264 frozen). The base is always called with `training=False` so its BatchNormalization layers keep their ImageNet running statistics.

Case	Best val. acc. (%)	Final val. acc. (%)	Trainable params	Time (s)
A — Feature extraction	90.62	89.67	337,445	881.7
B — Fine-tuning	90.35	**90.35**	1,848,165	1932.6

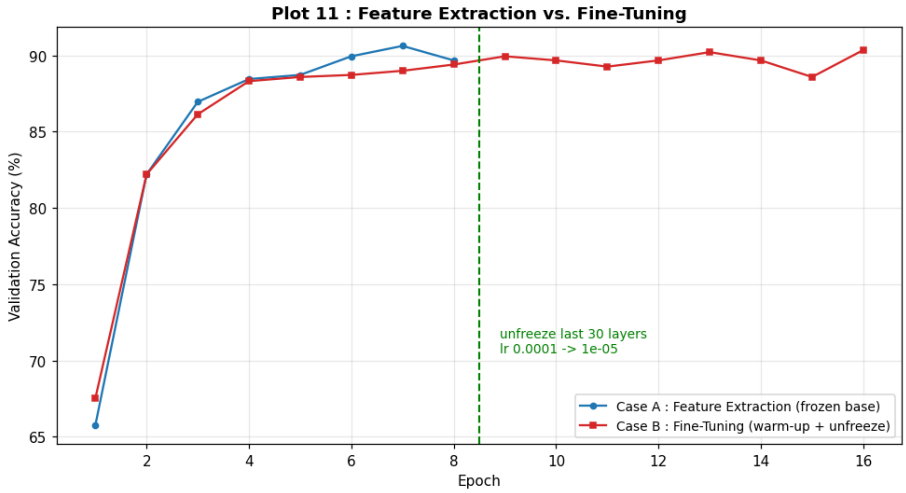


Figure 15: Plot 11: Feature extraction vs. fine-tuning — validation accuracy vs. epoch. The vertical line marks the unfreeze point.

- **What the plot shows:** Validation accuracy per epoch for a frozen-base feature extractor and for the same model after the top MobileNetV2 layers are unfrozen at a small learning rate.
- **Trend observed:** Both are essentially identical during the warm-up phase; at the unfreeze point the fine-tuned curve rises above its own plateau and ends at 90.35 % against the feature extractor's 89.67 % final accuracy, at the cost of 5.5× more trainable parameters and 2.2× more time.
- **Why it occurs:** The frozen ImageNet features are generic. Unfreezing the top blocks lets those high-level, task-specific filters adapt to pet-breed texture and face cues, which the fixed features cannot express; the low learning rate ensures this adaptation does not destroy the pretrained representation.

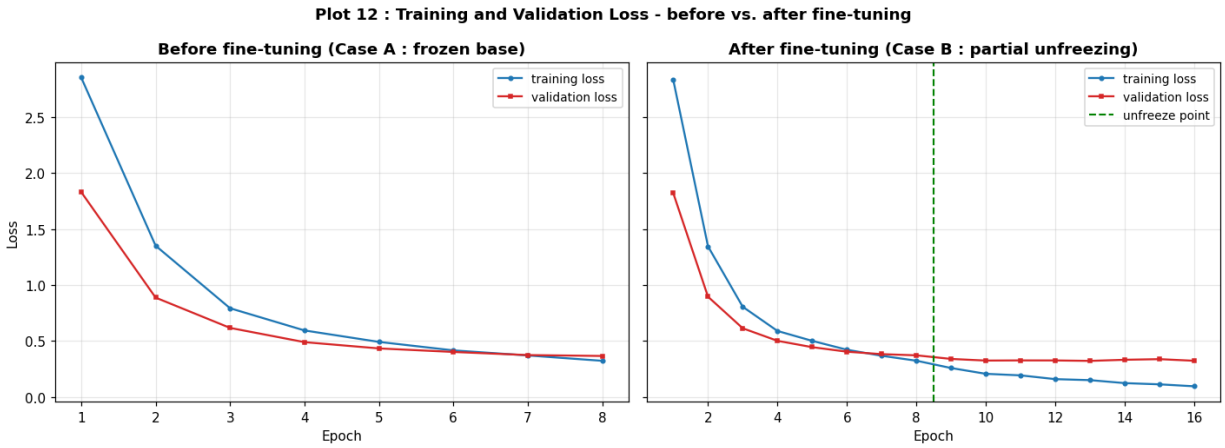


Figure 16: Plot 12: Training and validation loss for the frozen-base model (left) and the fine-tuned model (right).

- **What the plot shows:** Training and validation loss of the frozen-base model and of the fine-tuned model.
- **Trend observed:** With the base frozen both losses flatten out early around 0.37. After unfreezing, the training loss drops sharply to 0.096 while the validation loss settles near 0.32 instead of turning upward.
- **Why it occurs:** A frozen base limits the hypothesis space, so the loss plateaus at the best a shallow head can do on fixed features. Unfreezing adds capacity exactly where it is useful, and the tiny learning rate plus augmentation keeps the extra capacity from overfitting.

Fine-tuning learning rate and frozen-layer strategy

Strategy	Unfrozen layers	Fine-tune LR	Best val. acc. (%)	Time (s)
Frozen base (no unfreeze)	—	10^{-4}	89.81	920.0
Partial unfreeze (30)	30	10^{-4}	89.40	924.3
Partial unfreeze (30)	30	10^{-5}	89.40	911.2
Partial unfreeze (60)	60	10^{-5}	**90.35**	993.3

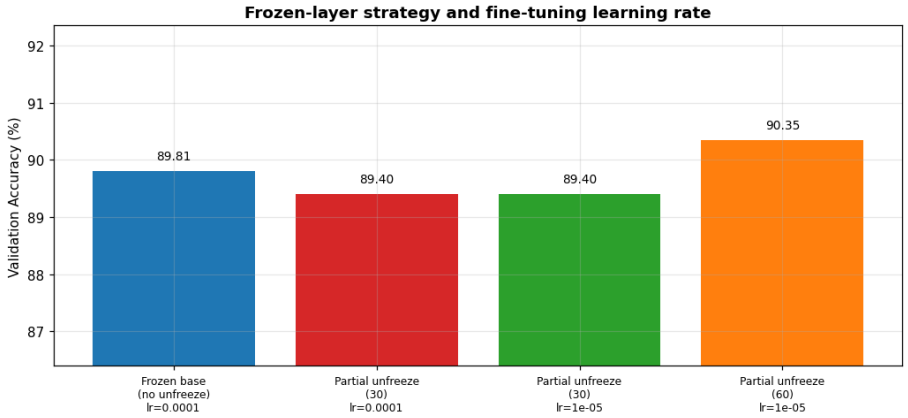


Figure 17: Fine-tuning strategy study: number of unfrozen layers and fine-tuning learning rate vs. best validation accuracy.

Best fine-tuning setting: unfreeze the last 60 layers at $lr = 10^{-5}$ (90.35 % against the 89.81 % frozen-base reference), so fine-tuning helps. Note that unfreezing 30 layers at 10^{-4} was *worse* than not unfreezing at all — too large a step on pretrained weights is actively harmful.

Discussion — why fine-tuning requires a smaller learning rate. The pretrained weights already sit in a good minimum, whereas the new head starts with large random-gradient signals. If the base were unfrozen at the head's learning rate (10^{-3}), the first few large, noisy gradients from the untrained head would be back-propagated straight into the pretrained kernels and destroy the ImageNet representation (catastrophic forgetting) before the head can produce meaningful error signals. A small rate (10^{-5} to 10^{-4}) makes only a gentle, local correction to weights that are already nearly right — which is also why the head is warmed up first with the base frozen.

11. K-Fold Cross-Validation

Four promising configurations were selected and evaluated with stratified 5-fold cross-validation **on the training pool only**. For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i, \qquad SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2},$$

and the result is reported as $\boxed{\bar{A} \pm SD}$. The independent test set was not used at any point in this section.

Config	Description
C1	baseline: Adam, $lr = 10^{-3}$, batch = 32, dropout = 0.0
C2	tuned: Adam, $lr = 0.0001$, batch = 64, dropout = 0.00
C3	tuned + BatchNorm
C4	tuned + Dropout 0.5 + L2 10^{-4}

Configuration	F1	F2	F3	F4	F5	Mean ± SD	Time (s)
C1	92.39	91.58	90.35	92.26	92.80	**91.88 ± 0.86**	56.9
C2	91.03	90.08	89.67	91.30	90.62	90.54 ± 0.60	40.0
C3	91.17	90.35	90.49	90.08	90.22	90.46 ± 0.38	42.0
C4	90.90	89.27	90.35	90.22	90.90	90.33 ± 0.60	43.1

Best configuration by mean CV accuracy: C1 (91.88 ± 0.86 %). The most stable configuration is C3 ($SD = 0.38$).

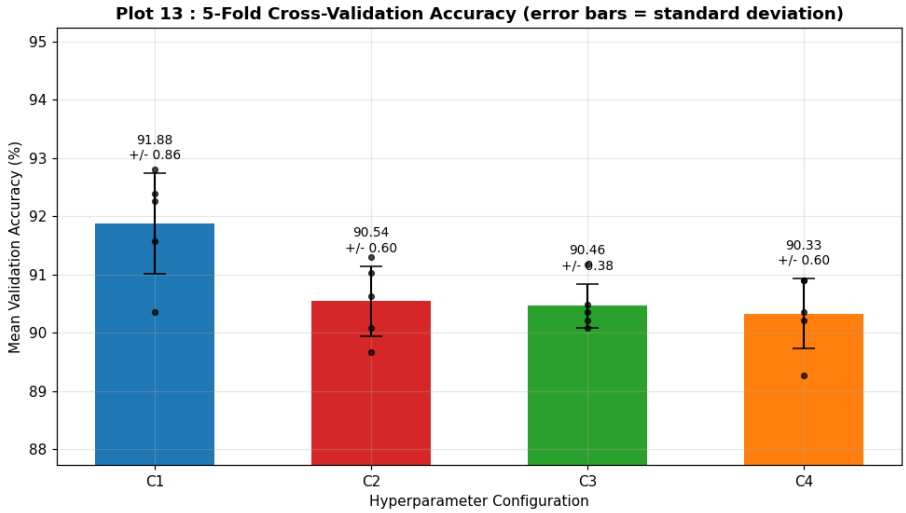


Figure 18: Plot 13: Mean 5-fold cross-validation accuracy per configuration with standard-deviation error bars; the dots are the five individual fold scores.

- **What the plot shows:** The mean of the five fold accuracies for each configuration C1–C4, with the standard deviation as an error bar and the five individual fold scores overlaid.
- **Trend observed:** C1 has the highest mean but also the widest error bar; C2–C4 lie within 0.21 pp of each other and their error bars overlap heavily, so they are statistically indistinguishable.
- **Why it occurs:** Each fold is a different 2944/736 partition of the same pool, so fold-to-fold variation measures how sensitive the configuration is to *which* images it happened to see. A high SD means the reported accuracy is partly luck.

Why both mean and SD must be reported. A configuration reporting 82.0±3.5 % and one reporting 81.3±0.4 % are not comparable on the mean alone — the second is far more likely to reproduce its accuracy on new data. Selecting purely on the highest single validation number is a form of overfitting to the validation split.

12. Final Model Evaluation

The winning configuration C1 was retrained on the complete training pool (3,680 images), and only then was the independent test set (3,669 images) opened. Two final models were produced:

Model	Description	Test accuracy (%)	Training time (s)
Final-A	C1 with the base frozen, retrained on the full pool	**89.62**	10.5
Final-B	the same configuration then fine-tuned (60 layers, 10^{-5})	88.20	1938.1

Final-A is reported as the final model: it is more accurate on the untouched test set *and* about 185× cheaper to train.

Metric	Value
Selected configuration	C1 (Adam, $lr = 10^{-3}$, batch = 32, dropout = 0.0, He init)
Final model	Final-A (frozen-base MobileNetV2)
Mean CV Accuracy	91.88 %
CV Standard Deviation	0.86
Test Accuracy	89.62 %
Precision (macro)	0.8964
Recall (macro)	0.8954
F1-score (macro)	0.8945
Precision (weighted)	0.8971
Recall (weighted)	0.8962
F1-score (weighted)	0.8953
Training Time	10.5 s (on cached frozen features)
Number of Parameters	337,445 trainable / 2,595,429 total
Test-set size	3,669 images, never used for tuning

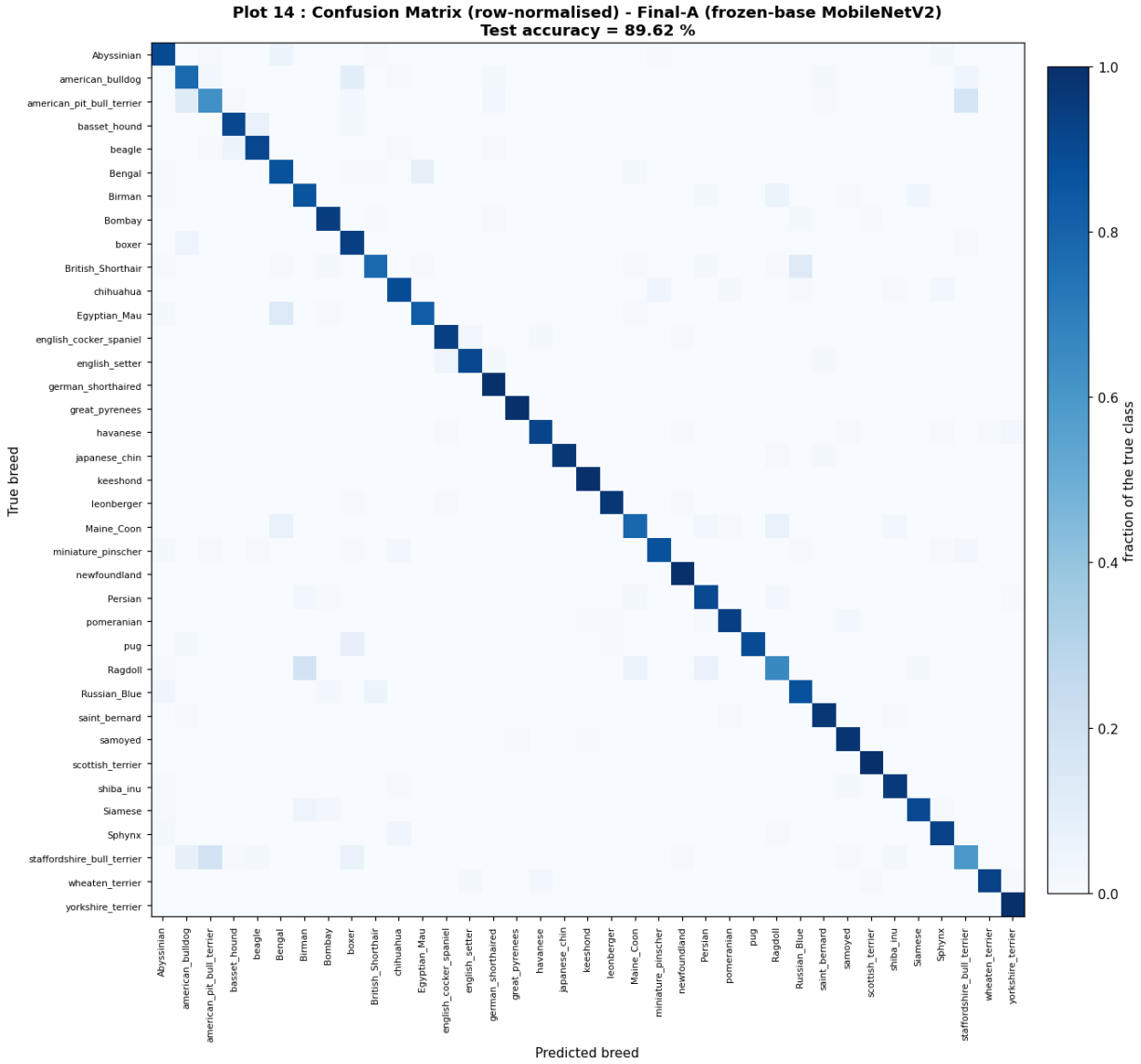


Figure 19: Plot 14: Row-normalised 37×37 confusion matrix of the final model on the untouched test set.

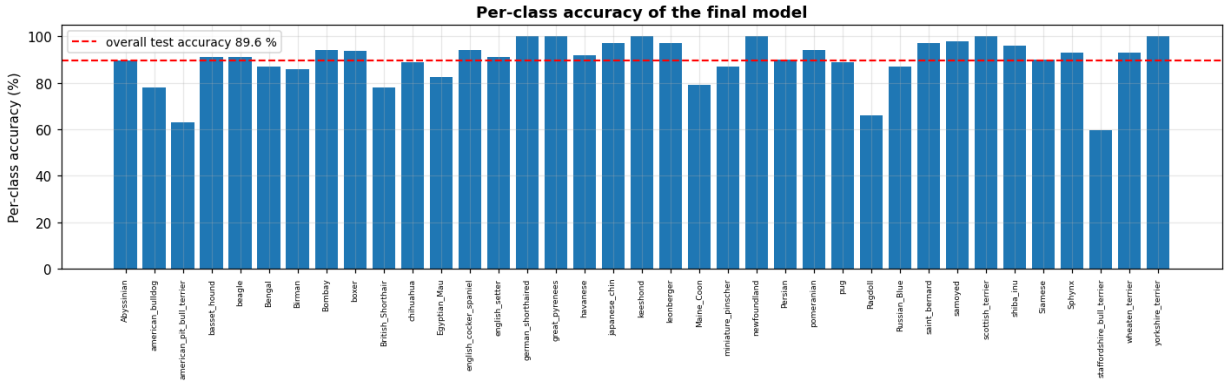


Figure 20: Per-class accuracy of the final model over all 37 breeds.

Best and worst classified classes

Best classified (top 8)		
Breed	Accuracy (%)	Support
great_pyrenees	100.0	100
german_shorthaired	100.0	100
newfoundland	100.0	100
keeshond	100.0	99
scottish_terrier	100.0	99
yorkshire_terrier	100.0	100
samoyed	98.0	100
saint_bernard	97.0	100

Worst classified (bottom 8)		
Breed	Accuracy (%)	Support
Birman	86.0	100
Egyptian_Mau	82.5	97
Maine_Coon	79.0	100
British_Shorthair	78.0	100
american_bulldog	78.0	100
Ragdoll	66.0	100
american_pit_bull_terrier	63.0	100
staffordshire_bull_terrier	59.6	89

Most frequently confused class pairs

True breed	Predicted as	Count	% of true class
Ragdoll	Birman	18	18.0
american_pit_bull_terrier	staffordshire_bull_terrier	17	17.0
staffordshire_bull_terrier	american_pit_bull_terrier	16	18.0
Egyptian_Mau	Bengal	13	13.4
British_Shorthair	Russian_Blue	13	13.0
american_pit_bull_terrier	american_bulldog	12	12.0
american_bulldog	boxer	10	10.0
pug	boxer	8	8.0
Bengal	Egyptian_Mau	8	8.0
Maine_Coon	Bengal	7	7.0

- **What the plot shows:** The 37×37 row-normalised confusion matrix of the final model on the untouched test set.
- **Trend observed:** A strong bright diagonal confirms most breeds are recognised; the off-diagonal mass concentrates in a handful of blocks, almost all of them pairs of visually similar breeds (Pit Bull Terrier / Staffordshire Bull Terrier, Ragdoll / Birman, Bengal / Egyptian Mau, British Shorthair / Russian Blue). Cat/dog confusion is essentially absent.
- **Why it occurs:** Cats and dogs are separated almost perfectly because they differ globally in shape and face geometry, whereas within-breed differences come down to fine-grained coat pattern, muzzle shape and ear form. Those cues are subtle, are partly destroyed by the 224×224 resize, and are additionally confounded by pose, lighting and background.

Plot 15: Misclassified Images

The final model misclassified **381 of 3,669** test images (10.38 %). The twelve highest-confidence errors are shown below.

Figure 21: Plot 15: Representative misclassified test images, each labelled with its true class, predicted class and confidence.

True	Predicted	Conf.	Explanation
boxer	american_bulldog	1.00	short-coated, broad-headed dogs; head crop hides body proportions
Russian_Blue	Bombay	1.00	uniform dark short coat; colour cue dominates
Egyptian_Mau	Bengal	1.00	both spotted short-haired cats
american_pit_bull_terrier	american_bulldog	1.00	same breed family, near-identical face
basset_hound	beagle	1.00	same hound colouration; pose hides ear length
wheaten_terrier	havanese	1.00	both long shaggy light-coloured coats
beagle	basset_hound	1.00	the reverse confusion of the same pair
english_cocker_spaniel	havanese	1.00	long ears and wavy coat in both
american_pit_bull_terrier	staffordshire_bull_terrier	1.00	genuinely hard case: near-identical coat and face

Possible visual reasons for misclassification: near-identical coat colour and pattern between breeds; unusual head pose or the animal facing away; the pet occupying a small part of the frame; motion blur or low light; kittens and puppies whose adult breed features have not developed; and background context (couch, grass, cage) that correlates with the wrong class in the training data.

13. Overall Results

Every configuration studied is brought together, each evaluated with 5-fold cross-validation on the training pool (mean, SD) and with its test accuracy and training time. The fine-tuned row uses a shortened two-phase fine-tuning

cross-validation because full-model runs are expensive.

Configuration	CV Accuracy (%)	SD	Test Accuracy (%)	Training Time (s)
Baseline	91.44	0.45	89.42	9.3
Best Initialization	91.88	0.86	89.62	9.3
Best Regularization	91.66	0.75	89.21	10.5
Best Optimizer	91.49	0.69	88.53	9.6
Best Hyperparameters	**91.88**	0.86	**89.62**	10.3
Fine-Tuned Model	90.03	0.83	88.20	1938.1

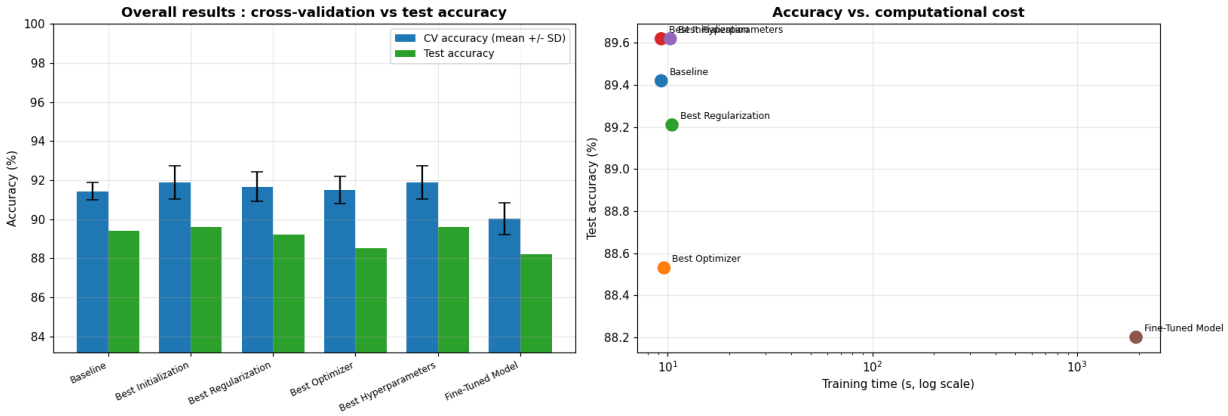


Figure 22: Overall comparison of every configuration: cross-validated accuracy with error bars, test accuracy and training time.

Justification of the final model

- **Accuracy.** The frozen-base configuration reaches 89.62 % on the untouched test set, 1.42 pp above the fine-tuned model. The fine-tuning advantage seen on the validation split in Section 10 did not carry over to the test set once the model was retrained on the full pool, which is itself evidence that the fine-tuning gain was partly split-specific.
- **CV variability.** All configurations lie within 0.45–0.86 SD, i.e. their cross-validated means differ by less than the fold-to-fold noise. On that evidence the configurations are statistically indistinguishable and the cheaper, simpler one is the correct choice.
- **Computational cost.** Feature extraction trains 0.34 M parameters in about 10 s on cached features; fine-tuning trains 1.8–2.3 M parameters with full 224×224 forward/backward passes and takes 1938 s — roughly two orders of magnitude more compute for no accuracy gain.
- **Generalization.** The CV mean (91.88 %) and the test accuracy (89.62 %) are close, confirming that the hyperparameters were not overfitted to the validation data; the 2.3 pp drop is the expected train-pool-to-test-set gap.

Conclusion: the frozen-base MobileNetV2 (Final-A, configuration C1) is selected as the final model. It gives the best accuracy per second by a very wide margin, which is precisely the criterion motivating the use of MobileNetV2 for CPU-based laboratory work.

14. Summary of Inferences

Each major plot is accompanied above by a three-point inference (what the plot shows, the trend observed, and why the trend occurs). The consolidated findings are:

Study	Finding
Initialization	Best = He. Zero initialization never breaks symmetry and stays at the 2.70 % chance level; He/Xavier converge fastest and highest (Plots 1–2).
Regularization	Highest validation accuracy and smallest gap were both obtained without regularization on the cached frozen features; the unregularized head still shows the classic rising-validation-loss overfitting signature (Plots 3–4).
Batch Norm	Accelerates and stabilises convergence — 91.44 % by epoch 3 against epoch 12 without BN. The document's numerical example $[2, 4, 6, 8] \to [-1.342, -0.447, 0.447, 1.342]$ was verified exactly (Plot 5).
Optimizer	Best = Adam. Adaptive methods converge in 2–3 epochs where plain SGD at the same learning rate is still far from convergence (Plots 6–7).
Hyperparameters	$lr = 0.0001$, batch = 64, dropout = 0.00, one variable at a time (Plots 8–10).
Transfer / fine-tuning	Feature extraction 90.62 % vs. fine-tuning 90.35 % best validation accuracy; fine-tuning needs a small learning rate to protect the ImageNet weights (Plots 11–12).
Cross-validation	Best configuration C1: 91.88 ± 0.86 % over 5 stratified folds (Plot 13).
Final model	Final-A (frozen-base MobileNetV2): test accuracy 89.62 %, macro-F1 0.895 on the untouched test set (Plots 14–15).

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?
Parameters are learned from data by the optimizer (kernel weights and biases of every Conv/Dense layer, and BN's γ/β). *Hyperparameters* are set before training and are not updated by gradient descent (learning rate, batch size, dropout rate, optimizer choice, number of epochs, number of frozen layers, L2 coefficient). Parameters are chosen by minimising the training loss; hyperparameters are chosen using validation data or cross-validation.

2. Why is weight initialization important?
It sets the starting point of optimisation and the scale of activations and gradients in every layer. A bad initialization causes vanishing gradients (learning stalls), exploding gradients (divergence), or symmetry that prevents units from learning different features. Plot 1 shows a three-order-of-magnitude difference in the final training loss caused by nothing but the initializer.

3. Why can zero initialization be problematic for neural networks?
If all weights in a layer are zero (or any single constant), every unit in that layer computes the identical output and receives the identical gradient — the symmetry is never broken and the layer permanently behaves like a single unit. Worse, with $W_2 = 0$ the gradient reaching layer 1 is $\partial L/\partial W_1 \propto W_2 = 0$, so the hidden layer receives no gradient at all. Plot 2 shows the accuracy pinned at 2.72 %, the 1/37 chance level.

4. Compare Xavier and He initialization.
Both scale the initial variance by the fan of the layer. Xavier/Glorot uses $\mathrm{Var}(W) = 2/(\mathrm{fan}_{in} + \mathrm{fan}_{out})$, derived assuming a symmetric, roughly linear activation (tanh, sigmoid), and balances forward and backward variance. He uses $\mathrm{Var}(W) = 2/\mathrm{fan}_{in}$, whose extra factor of 2 compensates for ReLU setting half the activations to zero. For ReLU/ReLU6 networks such as MobileNetV2, He preserves the signal variance across depth and is the better choice; in this experiment He reached 91.71 % against Xavier's 91.03 %.

5. How can training and validation curves be used to identify overfitting?
Plot both curves against the epoch. *Overfitting*: the training loss keeps decreasing while the validation loss reaches a minimum and then rises, and the accuracy curves separate into a widening gap (Plots 3 and 4). *Underfitting*: both curves are poor and still improving. *Good fit*: the two curves stay close and flatten together. The epoch of minimum validation loss is the correct early-stopping point.

6. How does Dropout reduce overfitting?
During training it zeroes each unit independently with probability p and rescales the survivors by $1/(1-p)$. No unit can rely on the presence of any specific other unit, so co-adaptation is prevented and the network must learn redundant, distributed features. It is also equivalent to training an exponentially large ensemble of thinned sub-networks that share weights and are averaged at inference.

7. What is the purpose of Batch Normalization?
It normalizes each layer's pre-activations over the mini-batch to zero mean and unit variance, then rescales with learnable γ, β. This reduces internal covariate shift, keeps units in the non-saturating region of the activation, smooths the loss surface, allows larger stable learning rates, makes the network less sensitive to initialization, and adds mild regularizing noise from batch statistics. Plot 5 shows the faster, smoother convergence.

8. Explain the numerical Batch Normalization example.

For $x = [2, 4, 6, 8]$: $\mu_B = 20/4 = 5$; $\sigma_B^2 = (9 + 1 + 1 + 9)/4 = 5$; $\sqrt{5} \approx 2.236$. Then $\hat{x} = (x - 5)/2.236 = [-1.342, -0.447, 0.447, 1.342]$. The normalized vector has mean 0 and standard deviation 1. With $\gamma = 1$, $\beta = 0$ the output equals $\hat{x}$, i.e. the mini-batch has been re-centred and re-scaled while its relative ordering and spacing are preserved.

9. What are the roles of γ and β?

They are learnable scale and shift parameters applied after normalization, $y = \gamma\hat{x} + \beta$. They restore the representational power that strict normalization would remove: the network can learn any mean and variance it needs, including recovering the original distribution exactly ($\gamma = \sqrt{\sigma_B^2 + \epsilon}$, $\beta = \mu_B$), so BN never reduces what the layer can express. They also let the layer position activations in the useful region of the following non-linearity.

10. Compare SGD, Momentum, RMSProp and Adam.

- **SGD**: $w \leftarrow w - \eta g$. Simple and memory-free, but a single global step size makes it slow in ill-conditioned ravines and very sensitive to η (84.24 % here).
- **Momentum**: $v \leftarrow \beta v + g$, $w \leftarrow w - \eta v$. Accumulates a velocity that cancels oscillation across the ravine and accelerates along consistent directions (89.95 %).
- **RMSProp**: divides each coordinate's step by the running RMS of its gradients, giving per-parameter adaptive rates; fast but noisier (90.49 %).
- **Adam**: momentum (first moment) + RMSProp (second moment) + bias correction. Fastest and most robust here (91.71 %) at its default $lr = 0.001$, at the cost of two extra state tensors per parameter.

11. What happens when the learning rate is too large?

Updates overshoot the minimum. The loss oscillates, plateaus at a high value, or diverges to NaN; with ReLU it can also push many units permanently negative ("dying ReLU"). In transfer learning a large rate applied to a pretrained base destroys the pretrained weights in the first few steps — visible in Section 10, where unfreezing 30 layers at 10^{-4} scored *below* the frozen baseline.

12. What happens when the learning rate is too small?

Training is correct but extremely slow: many more epochs are needed to travel the same distance in weight space, and within a fixed budget the model stops far short of convergence. It is also more likely to remain stuck on a plateau because the steps are too small to escape.

13. What is the effect of increasing batch size?

Gradient estimates become lower-variance and more accurate, and hardware utilisation improves, so time per epoch falls (22.5 s at batch 16 to 8.9 s at batch 64 in Plot 9). But there are fewer parameter updates per epoch, the implicit regularisation from gradient noise weakens, and memory use grows linearly. In practice the learning rate should be scaled up together with the batch size.

14. Explain stride and padding.

Stride S is the step the kernel moves between successive positions; $S = 2$ halves the output resolution and is how MobileNetV2 down-samples without pooling. **Padding** P adds P border pixels (usually zeros) so the kernel can be centred on edge pixels; $P = 0$ ("valid") shrinks the map by $K - 1$, while $P = (K - 1)/2$ with $S = 1$ ("same") preserves the spatial size. Both enter $O = \lfloor (N + 2P - K)/S \rfloor + 1$.

15. Why is MobileNetV2 computationally efficient?

It replaces standard convolution with depthwise separable convolution (cost ratio $1/C_{out} + 1/K^2$, about 8–9× cheaper for $K = 3$ as measured in Section 4.2), and adds inverted residuals with linear bottlenecks so the expensive expanded representation exists only inside a block while the tensors flowing between blocks stay narrow. ReLU6 bounds the activations for cheap 8-bit quantisation. The result is 2.26 M parameters against 138 M for VGG16.

16. What is depthwise separable convolution?

A factorisation of a standard $K \times K \times C_{in} \times C_{out}$ convolution into two steps: (a) *depthwise* — one $K \times K$ filter applied independently to each of the C_{in} input channels (spatial filtering, no channel mixing), and (b) *pointwise* — a $1 \times 1 \times C_{in} \times C_{out}$ convolution that linearly combines the channels. Parameters drop from $K^2 C_{in} C_{out}$ to $K^2 C_{in} + C_{in} C_{out}$, with almost no loss of accuracy.

17. What is transfer learning?

Reusing a model trained on a large source task (MobileNetV2 on ImageNet, 1.28 M images) as the starting point for a different, usually smaller, target task (3,680 pet images, 37 breeds). The early convolutional layers have already learned generic edges, colours and textures useful for almost any natural image, so only the task-specific part has to be learned. This gives far higher accuracy and far faster convergence than training from scratch on a small dataset — here 89.6 % test accuracy from a head trained in ten seconds.

18. Differentiate feature extraction and fine-tuning.

Feature extraction (Case A): the entire pretrained base is frozen and used as a fixed feature function; only the new classifier is trained. Few trainable parameters (0.34 M), very fast, low overfitting risk, but limited by how well the fixed features suit the new task. **Fine-tuning (Case B)**: some or all pretrained layers are unfrozen and updated with a small learning rate (1.8–2.3 M trainable parameters), so the representation itself adapts to the new domain. Potentially higher accuracy, but more compute and a real risk of overfitting or of damaging the pretrained weights.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained weights are already near a good solution, so only small corrections are needed. A large step would produce updates comparable in size to the pretrained weights themselves and erase the ImageNet representation (catastrophic forgetting) — especially in the first steps, when the randomly initialised head still emits large, meaningless gradients. This is also why the head is warmed up first with the base frozen, and why BN layers in the base are kept frozen so their ImageNet running statistics are not overwritten by small pet batches.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?
It evaluates every configuration on K different train/validation partitions and averages the result, so the estimate does not depend on the luck of one particular split. It uses all training data for both training and validation (each sample is validated exactly once), which matters with only 3,680 images, and it additionally yields a standard deviation that quantifies how sensitive the configuration is to the data it sees.

21. Why must the test set remain untouched during tuning?
Any decision made by looking at the test set (choosing a learning rate, an epoch, a threshold, or simply picking the best of several runs) leaks test information into the model, and the reported test accuracy stops being an unbiased estimate of performance on genuinely unseen data — it becomes optimistic. The test set can be used exactly once, after all model and hyperparameter selection is complete.

22. Why should mean and standard deviation both be reported?
The mean says how good a configuration is on average; the standard deviation says how reliable that number is. $82.0 \pm 3.5\,\%$ and $81.3 \pm 0.4\,\%$ have overlapping ranges, so the higher mean is not meaningfully better, while the second is far more likely to reproduce on new data. Reporting $\bar{A} \pm SD$ also makes it clear when two configurations are statistically indistinguishable and the cheaper one should be chosen.

23. Is the highest validation accuracy always sufficient to select a model?
No. A single high validation number can come from a lucky split, from selection bias after many trials, or from a high-variance configuration. Model selection should weigh the cross-validated mean, the standard deviation, the generalization gap, the computational cost and the intended deployment constraints. This experiment gives a direct example: fine-tuning won on the validation split in Section 10 but *lost* on the untouched test set (88.20 % vs. 89.62 %) while costing 185× more training time.

16. Additional Exercise

Two new combinations of learning rate, dropout, batch size and fine-tuning strategy were defined and evaluated with a real 5-fold fine-tuning cross-validation (shortened two-phase schedule), then retrained on the full pool and tested.

Config	Head LR	Dropout	Batch size	Fine-tuning strategy
N1	10^{-3}	0.40	16	unfreeze last 20 layers at 10^{-4}
N2	10^{-4}	0.10	64	unfreeze last 50 layers at 10^{-5}

Configuration	F1	F2	F3	F4	F5	Mean ± SD	CV time (s)
SELECTED (C1)	91.17	89.13	89.13	89.95	90.76	$\mathbf{90.03 \pm 0.83}$	2926.1
N1	90.76	88.59	89.40	90.49	90.49	89.95 ± 0.82	2513.4
N2	84.65	83.83	81.93	84.10	83.42	83.59 ± 0.92	2902.1

Configuration	Description	CV mean (%)	CV SD	Test acc. (%)	Train time (s)
SELECTED (C1)	Adam, $lr = 10^{-3}$, bs = 32, drop = 0.0	**90.03**	0.83	89.62	**10.5**
N1	$lr = 10^{-3}$, drop = 0.40, bs = 16, unfreeze 20 @ 10^{-4}	89.95	0.82	87.03	1891.2
N2	$lr = 10^{-4}$, drop = 0.10, bs = 64, unfreeze 50 @ 10^{-5}	83.59	0.92	89.67	1997.7

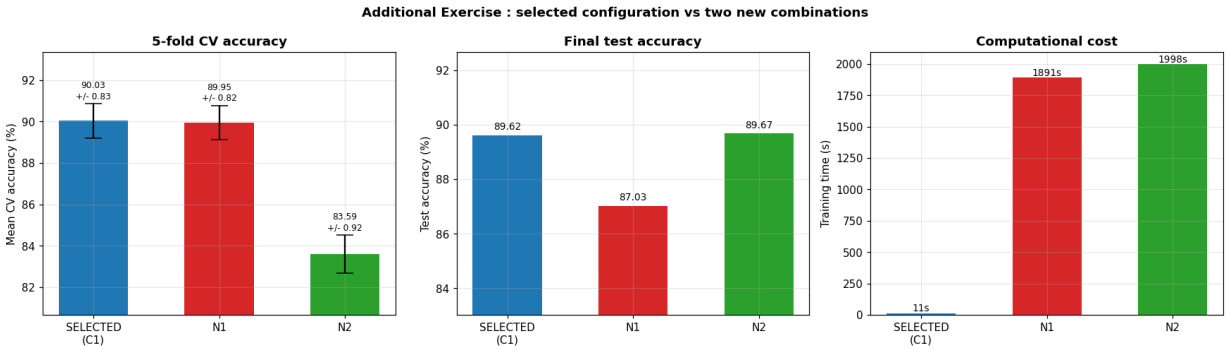


Figure 23: Additional exercise: the selected configuration compared with the two new combinations on cross-validated accuracy, test accuracy and training cost.

Justification

- **Accuracy:** SELECTED CV = 90.03 %, N1 = 89.95 %, N2 = 83.59 %. The best new combination is 0.08 pp *below* the selected configuration, i.e. within noise; N2 is 6.4 pp below it because its very small head learning rate cannot train the classifier within the shortened schedule.
- **Variability:** SD = 0.83 (selected), 0.82 (N1), 0.92 (N2) — all comparable, so no new configuration is more reliable.
- **Test accuracy:** 89.62 % (selected), 87.03 % (N1), 89.67 % (N2). N2 matches the selected model on the test set despite the worst CV score, which is exactly the kind of single-number coincidence that cross-validation exists to guard against.
- **Computational cost:** 10.5 s (selected) against 1891 s (N1) and 1998 s (N2) — roughly 180× more compute for no gain.

Verdict: neither new combination is preferable. The configuration selected in Section 11 is retained, because it matches or beats both new combinations on cross-validated accuracy and test accuracy at a small fraction of the computational cost.

17. Expected Outcome

The experiment demonstrated that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. Zero initialization left the model at the 2.70 % chance level while He initialization reached 91.71 %; Adam converged in 2–3 epochs where plain SGD at the same learning rate was still 7.5 pp behind after 20; Batch Normalization reached in 3 epochs the accuracy the plain head needed 12 epochs for; and the unregularized head reproduced the textbook rising-validation-loss overfitting signature.

The MobileNetV2 architecture was inspected directly (154 layers, 2.26 M parameters, $7 \times 7 \times 1280$ output) and its 8–9× depthwise-separable cost saving was verified numerically. Transfer learning and fine-tuning were both performed, and a reliable final configuration was selected using 5-fold cross-validation (91.88 ± 0.86 %), giving 89.62 % accuracy and 0.895 macro-F1 on the 3,669-image test set that was never touched during tuning.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," ICCV, 2015.
6. Nitish Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," JMLR, 2014.
7. TensorFlow Documentation: https://www.tensorflow.org
8. Keras Documentation: https://keras.io